ECA1408-Embedded System

Lab Experiments

-Aadhithya V(192511256)

Exp-1:

Code:

```
#define GAS_PIN A0
#define PIR_PIN 2
#define TRIG_PIN 8
#define ECHO_PIN 9
#define BUZZER 10
#define LED 11

void setup() {
  Serial.begin(9600);

  pinMode(PIR_PIN, INPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(LED, OUTPUT);
}

void loop() {
  int gas = analogRead(GAS_PIN);
  int motion = digitalRead(PIR_PIN);

  digitalWrite(TRIG_PIN, LOW);
```

```
delayMicroseconds(2);

digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);

digitalWrite(TRIG_PIN, LOW);

long duration = pulseIn(ECHO_PIN, HIGH);
float distance = duration * 0.0343 / 2;

if (gas > 500)
  digitalWrite(BUZZER, HIGH);
else
  digitalWrite(BUZZER, LOW);

digitalWrite(LED, motion);

Serial.print("Gas Level: ");
Serial.println(gas);

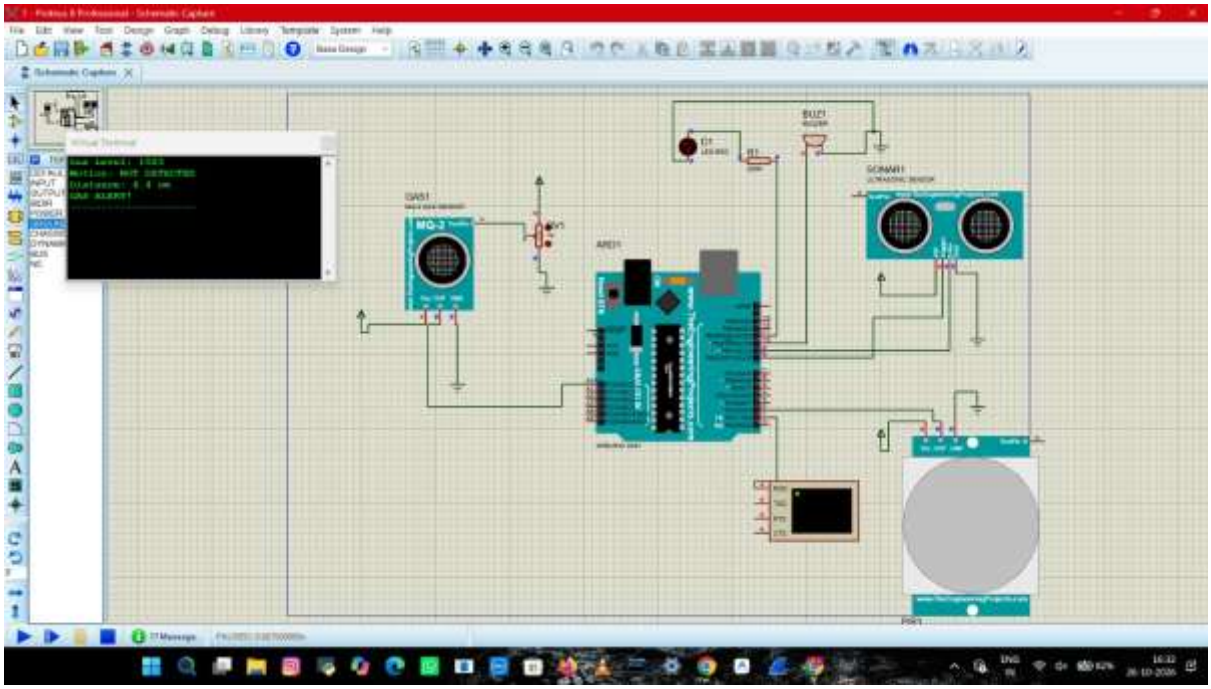
Serial.print("Motion: ");
if (motion)
  Serial.println("DETECTED");
else
  Serial.println("NOT DETECTED");

Serial.print("Distance: ");
Serial.print(distance, 1);
Serial.println(" cm");
```

```
  if (gas > 500)
    Serial.println("GAS ALERT!");
  else if (motion)
    Serial.println("OCCUPANCY DETECTED");
  else
    Serial.println("SYSTEM SAFE");

  Serial.println("--------------------");
  delay(1000);
}
```

Output:



Exp-2:

Code:

```
enum TrafficState {
  STATE_NS_GREEN,
  STATE_NS_YELLOW,
  STATE_EW_GREEN,
  STATE_EW_YELLOW
```

```
};

TrafficState currentState = STATE_NS_GREEN;

// North-South traffic lights
const int NS_RED_PIN = 2;
const int NS_YELLOW_PIN = 3;
const int NS_GREEN_PIN = 4;

// East-West traffic lights
const int EW_RED_PIN = 5;
const int EW_YELLOW_PIN = 6;
const int EW_GREEN_PIN = 7;

void setup() {
  pinMode(NS_RED_PIN, OUTPUT);
  pinMode(NS_YELLOW_PIN, OUTPUT);
  pinMode(NS_GREEN_PIN, OUTPUT);

  pinMode(EW_RED_PIN, OUTPUT);
  pinMode(EW_YELLOW_PIN, OUTPUT);
  pinMode(EW_GREEN_PIN, OUTPUT);

  Serial.begin(9600);
}

void allOff() {
  digitalWrite(NS_RED_PIN, LOW);
  digitalWrite(NS_YELLOW_PIN, LOW);
  digitalWrite(NS_GREEN_PIN, LOW);
```

```
  digitalWrite(EW_RED_PIN, LOW);
  digitalWrite(EW_YELLOW_PIN, LOW);
  digitalWrite(EW_GREEN_PIN, LOW);
}

void enterState(TrafficState state) {
  allOff();

  switch (state) {
    case STATE_NS_GREEN:
      digitalWrite(NS_GREEN_PIN, HIGH);
      digitalWrite(EW_RED_PIN, HIGH);
      Serial.println("STATE S0: NS GREEN - EW RED");
      break;

    case STATE_NS_YELLOW:
      digitalWrite(NS_YELLOW_PIN, HIGH);
      digitalWrite(EW_RED_PIN, HIGH);
      Serial.println("STATE S1: NS YELLOW - EW RED");
      break;

    case STATE_EW_GREEN:
      digitalWrite(NS_RED_PIN, HIGH);
      digitalWrite(EW_GREEN_PIN, HIGH);
      Serial.println("STATE S2: NS RED - EW GREEN");
      break;

    case STATE_EW_YELLOW:
      digitalWrite(NS_RED_PIN, HIGH);
```

```
    digitalWrite(EW_YELLOW_PIN, HIGH);
    Serial.println("STATE S3: NS RED - EW YELLOW");
    break;
  }
}

void loop() {
  switch (currentState) {
    case STATE_NS_GREEN:
      enterState(STATE_NS_GREEN);
      delay(10000);
      currentState = STATE_NS_YELLOW;
      break;

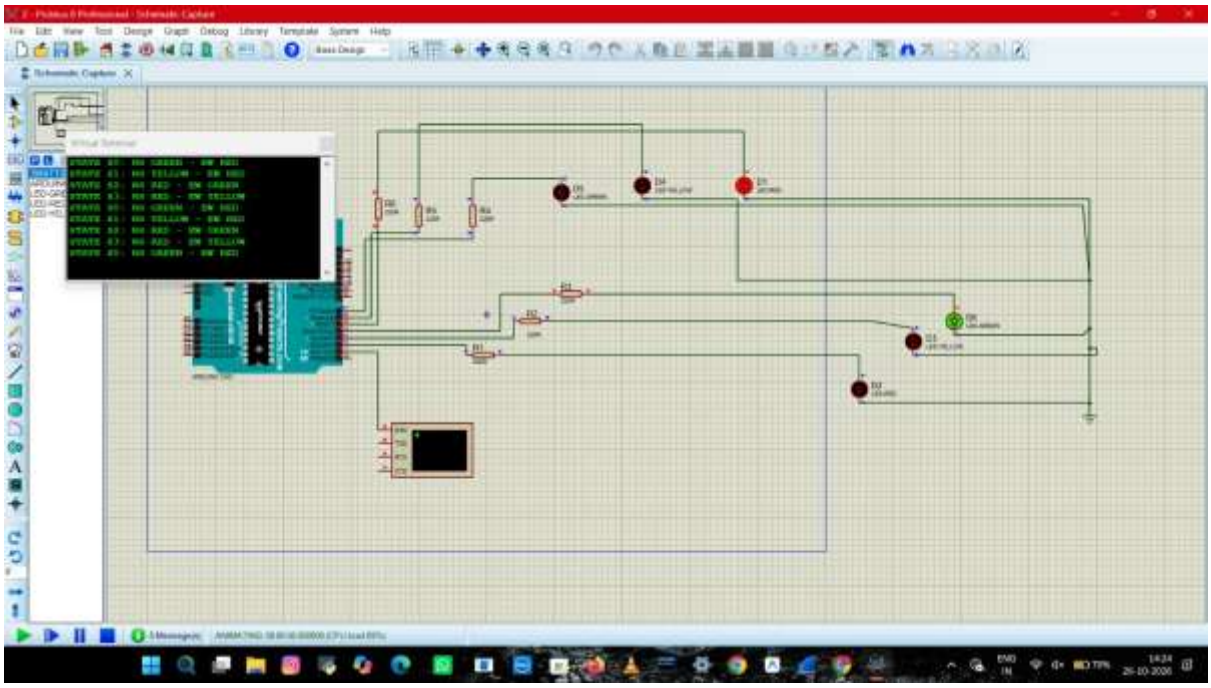
    case STATE_NS_YELLOW:
      enterState(STATE_NS_YELLOW);
      delay(3000);
      currentState = STATE_EW_GREEN;
      break;

    case STATE_EW_GREEN:
      enterState(STATE_EW_GREEN);
      delay(10000);
      currentState = STATE_EW_YELLOW;
      break;

    case STATE_EW_YELLOW:
      enterState(STATE_EW_YELLOW);
      delay(3000);
      currentState = STATE_NS_GREEN;
```

```
    break;
  }
}
```

Output:



Exp-3:

Code:

```
#include <Wire.h>

#define LM75_ADDRESS 0x48
#define PULSE_PIN 8

unsigned long pulseCount = 0;
unsigned long lastTempTime = 0;
unsigned long lastBpmTime = 0;

int previousPulseState = LOW;

float readTemperature() {
```

```
  Wire.beginTransmission(LM75_ADDRESS);
  Wire.write(0x00);
  if (Wire.endTransmission() != 0) {
    return -999.0;
  }

  Wire.requestFrom(LM75_ADDRESS, 2);

  if (Wire.available() >= 2) {
    byte msb = Wire.read();
    byte lsb = Wire.read();

    int16_t rawTemperature =
        (int16_t)(((uint16_t)msb << 8) | lsb);

    // LM75 temperature is a signed 11-bit value
    rawTemperature >>= 5;

    if (rawTemperature & 0x0400) {
      rawTemperature |= 0xF800;
    }

    return rawTemperature * 0.125;
  }

  return -999.0;
}

void setup() {
  Serial.begin(9600);
```

```
  Wire.begin();
  pinMode(PULSE_PIN, INPUT);

  Serial.println("I2C HEALTH MONITORING NODE");
  Serial.println("---------------------------");
}

void loop() {
  unsigned long currentTime = millis();

  // Count rising edges from the pulse sensor
  int currentPulseState = digitalRead(PULSE_PIN);

  if (currentPulseState == HIGH &&
      previousPulseState == LOW) {
    pulseCount++;
  }

  previousPulseState = currentPulseState;

  // Read temperature every 1 second
  if (currentTime - lastTempTime >= 1000) {
    lastTempTime = currentTime;

    float temperature = readTemperature();

    Serial.print("Temperature: ");

    if (temperature > -100) {
      Serial.print(temperature, 1);
```

```
      Serial.println(" C");
    } else {
      Serial.println("Sensor Error");
    }
  }

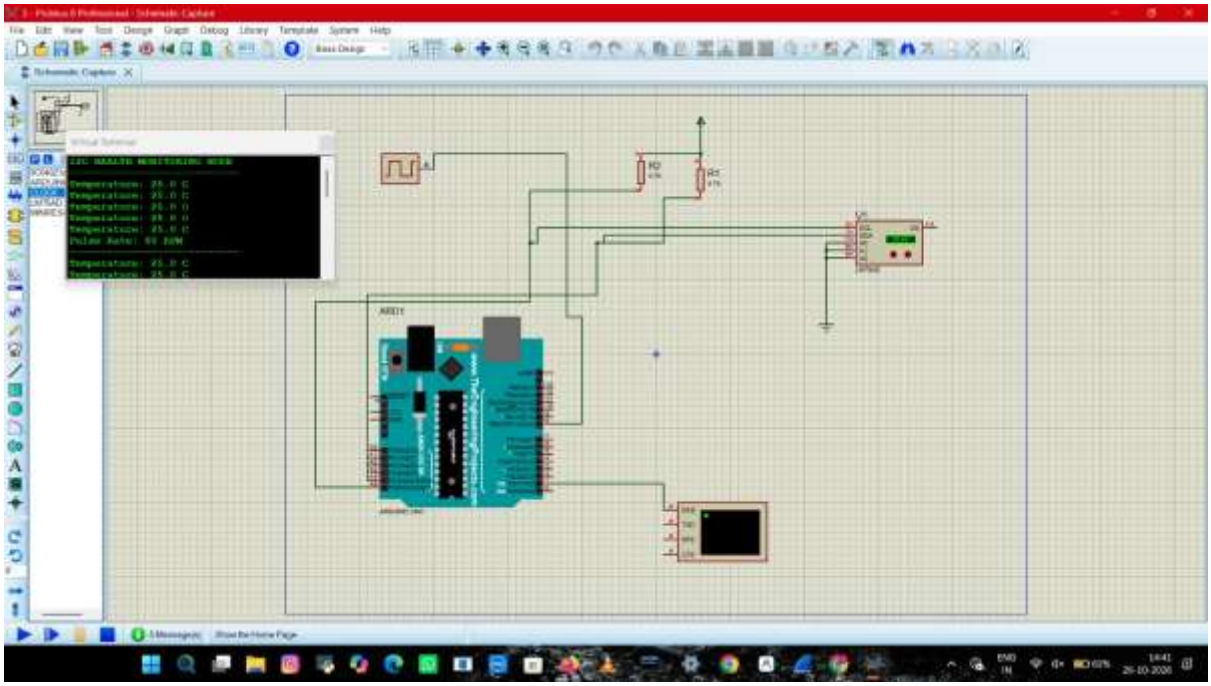
  // Calculate pulse rate every 5 seconds
  if (currentTime - lastBpmTime >= 5000) {
    lastBpmTime = currentTime;

    float bpm = pulseCount * 12.0;

    Serial.print("Pulse Rate: ");
    Serial.print(bpm, 0);
    Serial.println(" BPM");
    Serial.println("----------------------------");

    pulseCount = 0;
  }
}
```

Output:



Exp-6:

Code:

```
#define TEMP A0
#define FAN 8

volatile byte pwm = 0;
volatile byte count = 0;

ISR(TIMER1_COMPA_vect) {
  count++;

  if (count >= 100) {
    count = 0;
  }

  digitalWrite(FAN, count < pwm);
}
```

```
void setup() {
  Serial.begin(9600);
  pinMode(FAN, OUTPUT);

  noInterrupts();

  TCCR1A = 0;
  TCCR1B = (1 << WGM12) | (1 << CS11);
  OCR1A = 199;
  TIMSK1 = (1 << OCIE1A);

  interrupts();
}

void loop() {
  int adc = analogRead(TEMP);

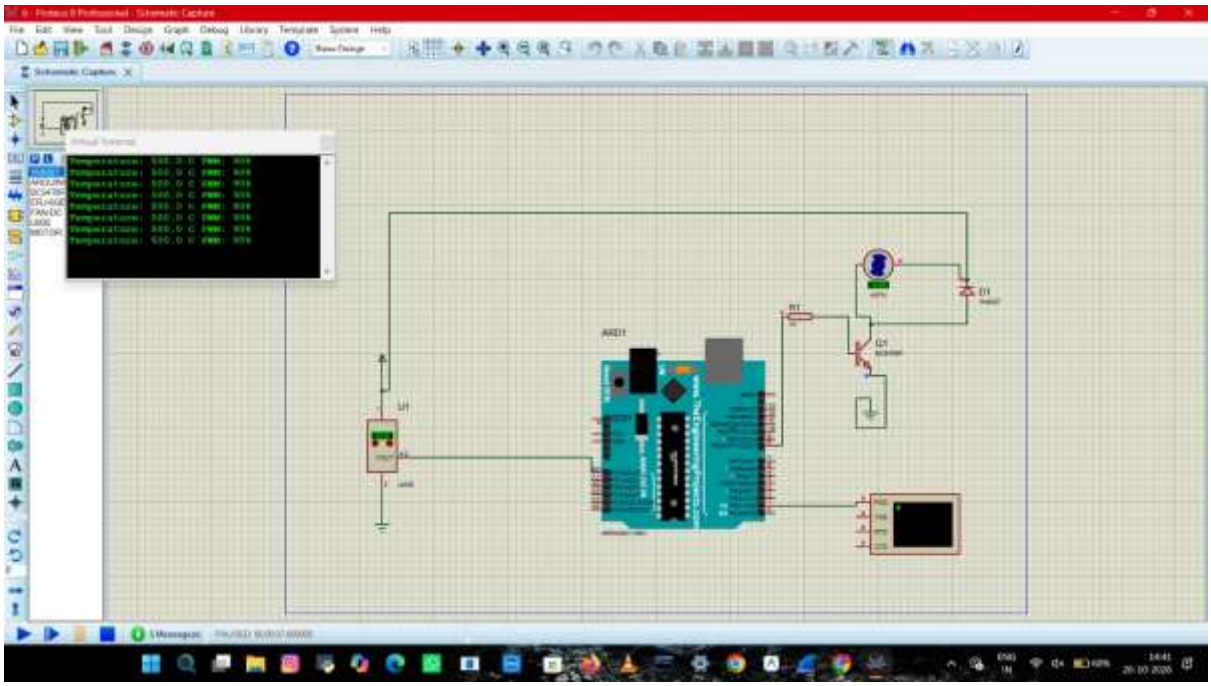
  // For an LM35 sensor powered at 5 V
  float temp = adc * 500.0 / 1023.0;

  if (temp < 25) {
    pwm = 0;
  } else if (temp < 30) {
    pwm = 30;
  } else if (temp < 35) {
    pwm = 60;
  } else {
    pwm = 90;
  }
```

```
  Serial.print("Temperature: ");
  Serial.print(temp, 1);
  Serial.print(" C PWM: ");
  Serial.print(pwm);
  Serial.println("%");

  delay(1000);
}
```

Ouput:



Exp-10:

Code:

```
#include <systemc.h>
#include <iostream>

using namespace sc_core;
using namespace std;
```

```
SC_MODULE(ClockExample)
{
   sc_in<bool> clk;

   void process()
   {
      while (true)
      {
         wait();
         cout << "Time: " << sc_time_stamp()
              << " - Clock Triggered" << endl;
      }
   }

   SC_CTOR(ClockExample)
   {
      SC_THREAD(process);
      sensitive << clk.pos();
   }
};

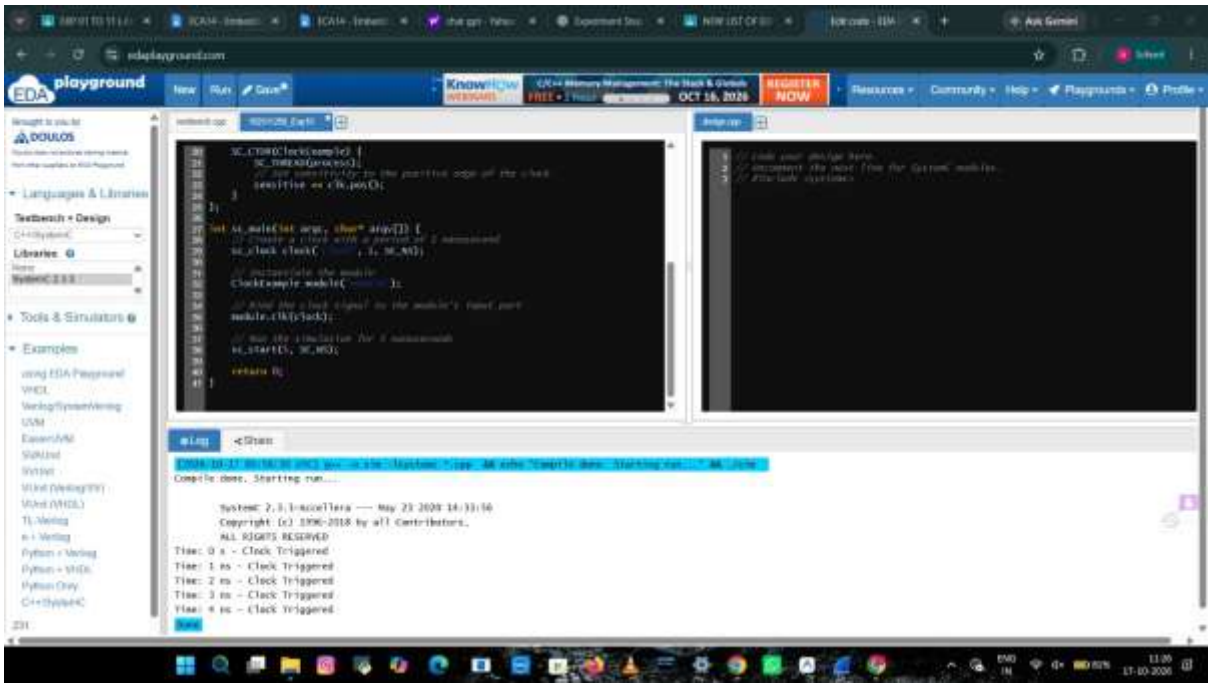
int sc_main(int argc, char* argv[])
{
   sc_clock clock("clock", 1, SC_NS);

   ClockExample module("module");
   module.clk(clock);

   sc_start(5, SC_NS);
```

```
    return 0;
}
```

Output:



Exp-11:

Code:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
using namespace std;

// Target Module
struct Target : sc_module
{
```

```
    tlm_utils::simple_target_socket<Target> socket;

    SC_CTOR(Target) : socket("socket")
    {
        socket.register_b_transport(this, &Target::b_transport);
    }

    void b_transport(tlm_generic_payload& trans, sc_time& delay)
    {
        int* data = reinterpret_cast<int*>(trans.get_data_ptr());

        cout << "Target received: " << *data << endl;

        delay += sc_time(10, SC_NS);
    }
};

// Initiator Module
struct Initiator : sc_module
{
    tlm_utils::simple_initiator_socket<Initiator> socket;

    SC_CTOR(Initiator) : socket("socket")
    {
        SC_THREAD(thread_process);
    }

    void thread_process()
    {
        tlm_generic_payload trans;
```

```
        sc_time delay = SC_ZERO_TIME;

        int data = 555;

        trans.set_command(TLM_WRITE_COMMAND);
        trans.set_data_ptr(
            reinterpret_cast<unsigned char*>(&data)
        );
        trans.set_data_length(sizeof(data));

        cout << "Initiator sending: " << data << endl;

        socket->b_transport(trans, delay);

        wait(delay);
    }
};

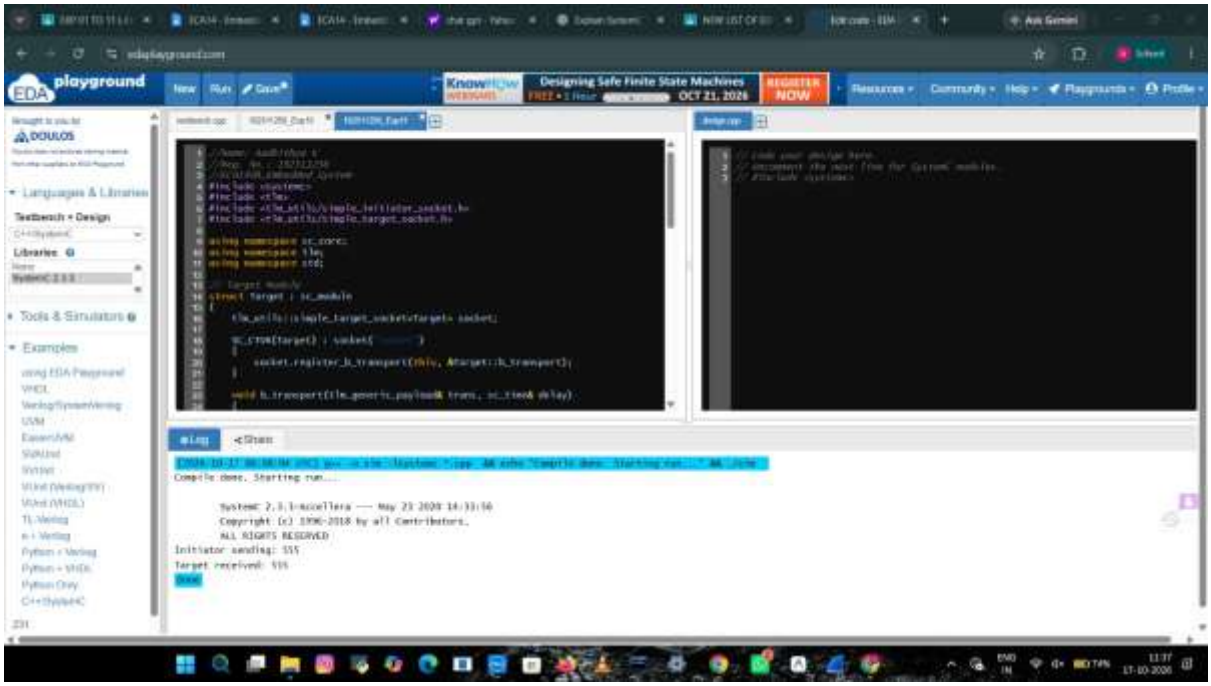
// Main Function
int sc_main(int argc, char* argv[])
{
    Initiator initiator("initiator");
    Target target("target");

    initiator.socket.bind(target.socket);

    sc_start();

    return 0;
}
```

Output:


playground
Designing Safe Finite State Machines
OCT 21, 2026
REGISTER NOW
Languages & Libraries
Tools & Simulators
Examples
Compile done. Starting run...
SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Initiator sending: 555
Target received: 555